TRABALHO PRÁTICO

DEIS – Integração de dados – 2024/2025

Lecionado por:

Anabela Borges Simões – abs@isec.pt

Realizado por:

Pedro Saraiva – a2023146226@isec.pt
Lucas Golobovante – a2023140728@isec.pt

Índice

1. Introdução 2

2. Interface Gráfica 10

2.1. Página principal 10

2.2. Manipulação ao ficheiro XML 12

2.3. Pesquisas ao ficheiro XML com XPath 14

2.4. Verificação com DTD 18

2.5. Verificação com XSD 19

2.6. Transformação com XSLT 20

2.7. Transformação com XQuery 26

3. Conclusão 28

1. Introdução

O objetivo do trabalho prático consiste em efetuar integração de dados provenientes destas duas fontes de dados e construir um modelo global de dados usando XML onde toda a informação pesquisada seja organizada em elementos/atributos, usando a hierarquia decidida pelos alunos como a mais correta para executar as tarefas propostas.

No trabalho foram abordadas as seguintes tecnologias:

- **XML** – Linguagem para estruturar, armazenar e transportar dados;
- **XPath** – Linguagem usada para navegar em documentos XML;
- **DTD** – Define a estrutura e regras de um documento XML;
- **XSD** – Esquema XML que define a estrutura e os tipos de dados;
- **XSLT** – Linguagem para transformar documentos XML em outros formatos;
- **XQuery** – Linguagem para consultar e manipular dados em XML, bem como transformar em outros formatos;

<u>Analisar as fontes de dados (S):</u>

As duas principais fontes de dados deste trabalho prático são as páginas web dos sites: https://pt.wikipedia.org/wiki/ e https://pt.db-city.com/, tal como já foi dito anteriormente.

O nosso projeto extrai informação dos seguintes dados:

- Nome do país
- Capital do país
- Continente do país
- Link para a imagem da bandeira do país
- Área do país
- Nº de habitantes do país
- Densidade populacional o país (nº de habitantes por km^2)
- Presidente/Chefe de estado
- Nº de casos de COVID registados
- Nº de mortes de COVID registados **(opcional)**
- Moeda **(opcional)**
- Hino **(opcional)**
- Língua(s) oficial(ais) do país
- Lista de Religiões
- Lista de cidades importantes
- Lista de países de fronteira

Nós decidimos abordar a extração de dados de uma forma robusta, isto é, tendo sempre um segundo plano para quando algo falhasse/não era encontrado. Primeiramente, nós extraímos os dados/atributos de ambas as fontes de dados e não de apenas uma, sendo assim necessário para cada atributo a extrair das páginas web, criar um Wrapper para o site Wikipédia, um Wrapper para o site Db-City, e ainda uma outra função de junção destes mesmos Wrappers, de forma a implementar e guardar mais tarde no ficheiro XML uma única informação aprimorada.

De tal forma que, para todos os atributos a extrair a pedido no enunciado e ainda aqueles criados por nós, haverá sempre no ficheiro "Wrappers.java" 3 funções dedicadas a cada um destes mesmos dados (não aplicando aos atributos onde têm informação apenas num dos sites, sendo necessária apenas 1 função/Wrapper para o atributo).

Esta terceira função, a qual é aplicada à maioria dos atributos, tal como já foi falado anteriormente tem sempre um dos dois Wrappers predefinido, isto é, nós ao testar os dois Wrappers de cada atributo vimos qual seria mais viável, não só por não divergir tanto no site de país para país, mas também pela forma de extrair ser mais simples, poupando tempo na execução de extração dos dados. Caso esse Wrapper predefinido na função seja *null* (não tenha valor nenhum, por não ter encontrado nada no site) então será usado o outro Wrapper (extraindo a mesma informação no outro site) e se ainda assim, este segundo Wrapper também for *null*, a função devolve "Desconhecido(a)" como informação para aquele atributo, de forma a não dar erro, caso a informação inserida pelo utilizador não for propriamente um país.

Segue um exemplo das 3 funções faladas anteriormente:

```java
// Funções para obter os hinos
static String obtem_hino_wiki(String pesquisa){
    String link = "https://pt.wikipedia.org/wiki/";
    try {
        HttpRequestFunctions.httpRequest1(link, pesquisa, "./sources/" + pesquisa + "_wiki.html");

        String linha;
        String erl = "<b>Hino:</b>(?:.+)data-mwtitle=\"(?:[^\\.]+)\\.(?:.+)<source src=\"([^\"]+)\"";
        Pattern pl = Pattern.compile(erl);

        Scanner in = new Scanner(new FileInputStream("./sources/" + pesquisa + "_wiki.html"));

        while (in.hasNextLine()) {
            linha = in.nextLine();
            Matcher nl = pl.matcher(linha);
            if(nl.find()) {
                in.close();
                return nl.group(1);
            }
        }
        in.close();
        return null;
    } catch (FileNotFoundException e) { // 'pesquisa' nao existe no URL do site: Wikipedia
        System.out.println("\n");
        System.out.println("Erro acessando a Wikipedia com " + pesquisa);
        return null;
    } catch (IOException e) { // outros erros à cerca do site: Wikipedia
        System.out.println("\n");
        System.out.println("Erro acessando a Wikipedia com " + pesquisa + ": " + e.getMessage());
        return null;
    }
}
```

```java
static String obtem_hino_dbcity(String pesquisa){
    String link = "https://pt.db-city.com/";
    try {
        HttpRequestFunctions.httpRequest1(link, pesquisa, "./sources/" + pesquisa + "_dbcity.html");

        String linha;
        String erl = "<th>Hino\\snacional\\s<span(?:.+)<td>(?:[^<]+)<br/><audio\\scontrols><source\\ssrc=\"([^\"]+)\"";
        Pattern pl = Pattern.compile(erl);

        Scanner in = new Scanner(new FileInputStream("./sources/" + pesquisa + "_dbcity.html"));

        while (in.hasNextLine()) {
            linha = in.nextLine();
            Matcher nl = pl.matcher(linha);
            if(nl.find()) {
                in.close();
                return nl.group(1); // link do hino
            }
        }
        in.close();
        return null;
    } catch (FileNotFoundException e) { // 'pesquisa' nao existe no URL do site: DBCity
        System.out.println("\n");
        System.out.println("Erro acessando a DBCity com " + pesquisa);
        return null;
    } catch (IOException e) { // outros erros à cerca do site: DBCity
        System.out.println("\n");
        System.out.println("Erro acessando a DBCity com " + pesquisa + ": " + e.getMessage());
        return null;
    }
}
```

```java
static String obtem_hino(String pais) throws IOException { // Justificação da priorização do site Db-City: está-se a prioriza
    String hinoWiki = obtem_hino_wiki(pais);
    String hinoDbCity = obtem_hino_dbcity(pais);

    if (hinoDbCity != null && !hinoDbCity.isBlank()) {
        return hinoDbCity;
    } else if (hinoWiki != null && !hinoWiki.isBlank()) {
        return hinoWiki;
    } else {
        return "desconhecida";
    }
}
```

Definir o esquema global (G):

O nosso ficheiro principal, onde é guardada a informação geral de todos os países chama-se: “países.xml”, eis a estrutura do mesmo já sendo validado por DTD e XSD:

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE lista SYSTEM "paises.dtd">
<lista xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="paises.xsd">
    <pais nome="Romênia">
        <capital>Bucareste</capital>
        <continente>Europa</continente>
        <img_bandeira>https://dwpt1kkww6vki.cloudfront.net/img/drapeau/120/174.png</img_bandeira>
        <area>238391.0</area>
        <habitantes>19524000</habitantes>
        <habitantes_porKM>81</habitantes_porKM>
        <presidente>Klaus Iohannis</presidente>
        <casos_covid>958918</casos_covid>
        <mortes_covid>23674</mortes_covid>
        <moeda>Leu romeno</moeda>
        <hino>https://db-city.s3.amazonaws.com/anthem/174.mp3</hino>
        <linguas>
            <lingua>Língua romena</lingua>
        </linguas>
        <religioes>
            <religiao>Ortodoxo - 70 %</religiao>
            <religiao>Cristianismo - 6 %</religiao>
            <religiao>Protestantismo - 6 %</religiao>
        </religioes>
        <cidades_importantes>
            <cidade>teste1</cidade>
        </cidades_importantes>
        <paises_fronteira>
            <pais_fronteira>teste2</pais_fronteira>
        </paises_fronteira>
    </pais>
</lista>
```

Implementação dos Wrappers (M):

Capital:

Wrapper Wikipédia: obtem_capital_wiki(String pesquisa) – Primeiramente o Wrapper vai passado linhas à frente (*while (in.hasNextLine())*) até encontrar a primeira expressão regular que no caso é a tag “Capital”, logo após dar *match* a esta primeira ER o Wrapper volta a passar linhas à frente até encontrar a segunda expressão regular onde basicamente extrai o nome da Capital.

Wrapper Db-City: obtem_capital_dbcity(String pesquisa) – Apenas encontra a linha que tem a capital e extrai a mesma graças à única expressão regular implementada neste Wrapper.

Função de decisão: obtem_capital(String pais) – O Wrapper predefinido é o da Wikipédia sem razão específica.

Continente:

Wrapper Wikipédia: obtem_continente_wiki(String pesquisa) – Procura a expressão regular `"continente\s([A-Za-zÀ-ÿ]+)"` e extrai o nome do continente através do grupo criado na ER.

Wrapper Db-City: obtem_continente_dbcity(String pesquisa) – Usa a expressão `"<th>Continente</th><td><a href=\"/([a-zA-ZÀ-ÿ]+)\"` para obter o nome através do grupo criado na ER.

Função de decisão: obtem_continente(String pais) – Prioriza a DB-City pela consistência dos dados.

Bandeira:

Wrapper Wikipédia: obtem_bandeira_wiki(String pesquisa) – Encontra a expressão `<a\shref="([/A-Za-zÀ-ÿ\(\)\.:_]+)"...title="Bandeira...">` para obter parte do link relativo através do grupo criado na ER, em seguida retorna o link através de return "https://pt.wikipedia.org" + n1.group(1);.

Wrapper Db-City: obtem_bandeira_dbcity(String pesquisa) – Usa a expressão `title="Bandeira..."><img src="([a-zA-ZÀ-ÿ:_\./0-9]+)` para extrair o link direto através do grupo criado na ER.

Função de decisão: obtem_bandeira(String pais) – Prioriza a DB-City por fornecer o link direto da imagem.

Línguas Oficiais:

Wrapper Wikipédia: obtem_linguas_wiki(String pesquisa) – Procura a tag `<b>Língua oficial</b>` em seguida num ciclo while adiciona as informações desejadas num Arraylist.

Wrapper Db-City: obtem_linguas_dbcity(String pesquisa) – Encontra inicialmente `"<th>Língua oficial</th><td>"` em seguida pega as informações das línguas que terminam cada uma com a tag ` "<br/>"` até que chege a tag ` "</td>"`.

Função de decisão: obtem_linguas(String pais) – Prioriza a DB-City pela precisão e formato estruturado.

Área:

Wrapper Wikipédia: obtem_area_wiki(String pesquisa) – Procura inicialmente a ER `"\">Área(?:</a>)?"` em seguida a ER ` "</th></tr>"` e a seguir a informação desejada é encontrada por uma terceira ER, após encontrar essa informação passa por um processo de substituição de caracteres. Como se pode observar, neste Wrapper há 3 possibilidades de grupo/busca na terceira ER, um com o espaço ( ) no código HTML simbolizado com: “ ”, outro com “ ” e ainda outro com “.”. De tal forma que primeiramente o Wrapper vê qual destas opções é a que está a ser usada na página HTML do país em questão, depois lida com a situação e elimina todos esses caracteres até ficar no formato desejado.

Wrapper Db-City: obtem_area_dbcity(String pesquisa) – Usa a expressão `">Superfície</a></th><td>([0-9\.]+)` para extrair o número através do grupo criado na ER.

Função de decisão: obtem_area(String pais) – Prioriza a Wikipédia por apresentar casas decimais e maior detalhe.

Habitantes:

Wrapper Wikipédia: obtem_habitantes_wiki(String pesquisa) – Procura `">População(?:</a>)?"` e extrai a informaçãoo desejada com os grupos da ER `\d+(?: \d+)+` ou `\d[\d\s]+`.

Wrapper Db-City: obtem_habitantes_dbcity(String pesquisa) – Usa `">População</a></th><td>([0-9\.]+)` para obter o valor.

Função de decisão: obtem_habitantes(String pais) – Prioriza a DB-City por melhor uniformidade dos dados.

Presidente:

Wrapper Wikipédia: obtem_presidente_wiki(String pesquisa) – Procura `>Presidente</a>` em seguida obtem a informação desejada através da ER `>([^<]+)</a>`.

Wrapper Db-City: obtem_presidente_dbcity(String pesquisa) – Através da única ER consegue encontrar primeiramente `Chefe de estado <span` e depois obtém aa informação desejada.

Função de decisão: obtem_presidente(String pais) – Prioriza a DB-City pela consistência entre países.

Religiões:

Wrapper Wikipédia: obtem_religiao_wiki(String pesquisa) – Usa `<b>Religião</b>` em seguida a ER `>.+>([^<]+)</a>` para extrair a informação.

Wrapper Db-City: obtem_religiao_dbcity(String pesquisa) – Encontra inicialmente a ER ` "\">Religião <span"` em seguida extrai a informação desejada terminada com a tag ` "<li>"`e adiciona no Arraylist até encontrar a tag ` "</ol>"`.

Função de decisão: obtem_religioes(String pais) – Prioriza a DB-City por apresentar percentagens e melhor formatação.

Cidades Importantes:

Wrapper Wikipédia: obtem_cidades_wiki(String pesquisa) – Procura bloco de cidades com `<td rowspan=...>` e extrai desse bloco com `>([azAZÀÿ\s]+)</a></b></td>`.

Wrapper Db-City: obtem_cidades_dbcity(String pesquisa) – Extrai do bloco `<div id="block_bigcity...">` com `title="([^"]+)">`.

Função de decisão: obtem_cidades_importantes(String pais) – Prioriza a Wikipédia por limitar consistentemente a 20 cidades.

Fronteiras:

Wrapper Wikipédia: obtem_fronteiras_wiki(String pesquisa) – Encontra `<a>Fronteira</a>` e extrai os títulos com `title="([^"]+)"` para o arraylist.

Wrapper Db-City: obtem_fronteiras_dbcity(String pesquisa) – Extrai do bloco `<div id="block_border">` com `title="([^"]+)"`.

Função de decisão: obtem_fronteiras(String pais) – Prioriza a Wikipédia por maior exatidão geográfica.

Covid-19:

Wrappers Db-City:

obtem_casos_covid – busca `<th>Confirmado` na ER e na mesma extrai os casos que depois são formatados para a forma que se pretende apresentar.

obtem_mortes_covid(String pesquisa) – busca `<th>Morte<` na ER e depois na mesma extrai o numero de mortes.

Moeda:

Wrapper Wikipédia: obtem_moeda_wiki(String pesquisa) – Procura `><b>Moeda</b>` em seguida extrai a informação por `"title=\"[^\"]+\">([^<]+)<\\/a>" `.

Wrapper Db-City: obtem_moeda_dbcity(String pesquisa) – Usa `title="Moeda...">([^<]+)</a>` extraindo da própria ER a informação desejada.

Função de decisão: obtem_moeda(String pais) – Ambas funcionam bem; preferência arbitrária pela Wikipédia.

Hino:

Wrapper Wikipédia: obtem_hino_wiki(String pesquisa) – Extrai o link do hino pela própria ER `<b>Hino:</b>...<source src="([^"]+)`.

Wrapper Db-City: obtem_hino_dbcity(String pesquisa) – Extrai de `<audio...<source src="([^"]+)`.

Função de decisão: obtem_hino(String pais) – Prioriza a DB-City por fornecer links mais diretos.

2. Interface Gráfica

2.1. Página principal

O ficheiro que está encarregue da interface visual do projeto chama-se: "interfaceGrafica.java".

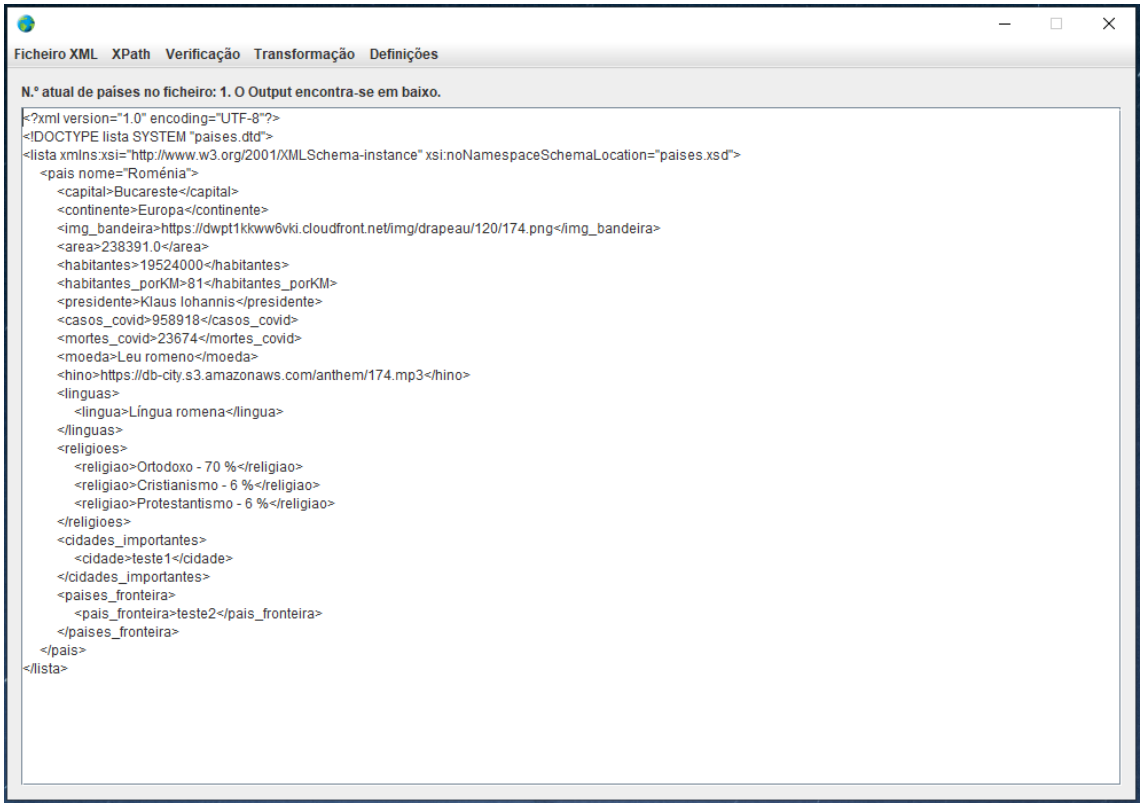
Quando é aberta a aplicação, é logo ilustrado o conteúdo do ficheiro "paises.xml" na caixa de texto principal de forma a o utilizador não ter de estar sempre a clicar num botão para ver o conteúdo do mesmo. No topo desta caixa de texto encontra-se também uma *label* que ilustra o número de países guardados no ficheiro a tempo real, isto é, sempre que é adicionado um país ou até mesmo removido essa mesma *label* é atualizada em conformidade com o número de países que é alterado no ficheiro (esta pesquisa está a ser feita com XPath).

Também se pode observar que no topo da página principal da aplicação encontra-se um menu com várias opções, tanto de manipulação ao ficheiro XML, como de pesquisas XPATH ao mesmo, como também a verificação feita com DTD e XSD, transformação com XLST e XQuery e ainda uma aba para as definições.

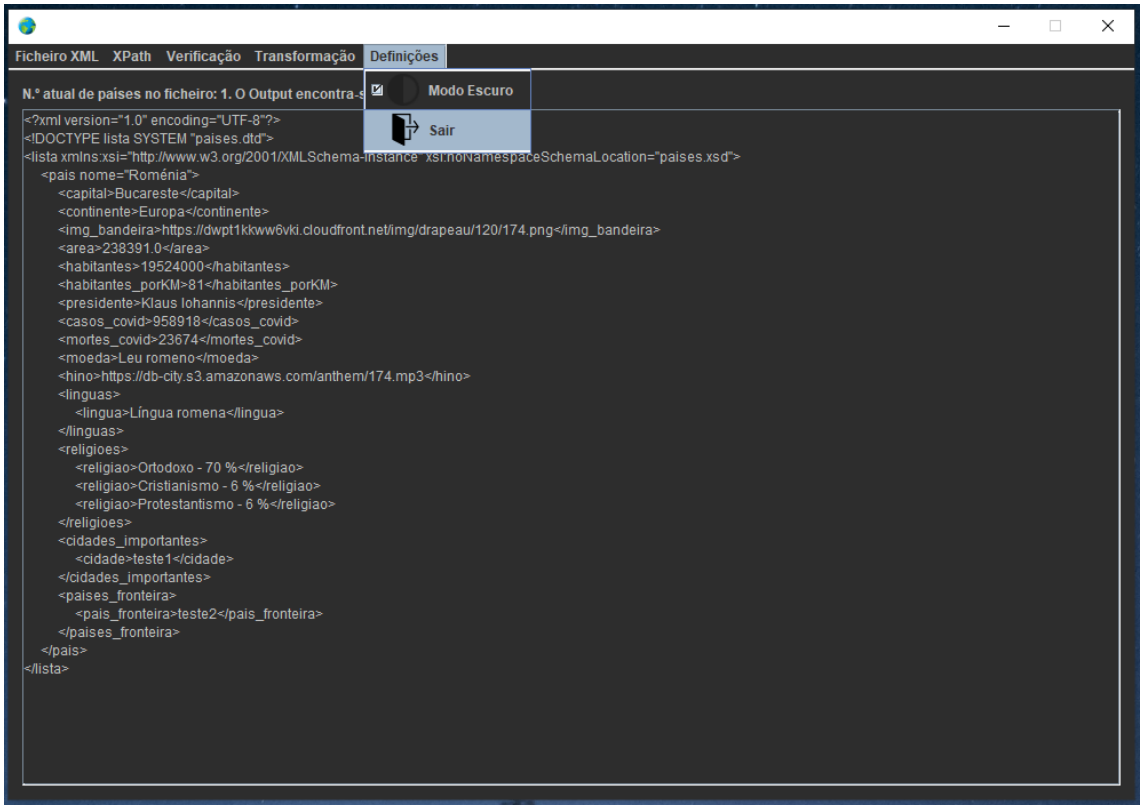
É importante também referir que sempre que há uma alteração a um atributo de qual que seja o país e até mesmo sempre que se adiciona ou remove valores a listas (atributos arrays) a caixa de texto principal (onde está a ser ilustrado o conteúdo do ficheiro XML) é sempre atualizada de forma a o utilizador não ter que clicar em nenhum botão para observar o que alterou.

O nosso projeto trata de todos os incidentes que podem vir a acontecer, isto é, sempre que um utilizador não insere algo numa caixa de texto e tenta ver o resultado/alterar algo o programa não deixa avançar, mostrando uma mensagem específica visual de erro ao utilizador de forma a o mesmo estar ciente do quê em concreto que deu errado. Isto funciona nas seguintes situações: quando não é inserido algo numa certa caixa de texto (como já foi falado anteriormente), quando o que é inserido não é o suposto (por exemplo numa das transformações em XSLT e XQuery onde é suposto o utilizador inserir uma letra e observar os países que começam por essa letra, se o mesmo não inserir apenas e só uma letra e não outro tipo de caracter aparece uma mensagem de erro ao utilizador a indicar o problema, também é lhe apresentada uma mensagem de erro quando o programa não encontra aquilo que o utilizador deseja, seja o nome de um país, o resultado duma pesquisa XPath, etc.

Em seguida ilustra-se as diferentes opções do menu superior, bem como a explicação de cada uma dessas opções:

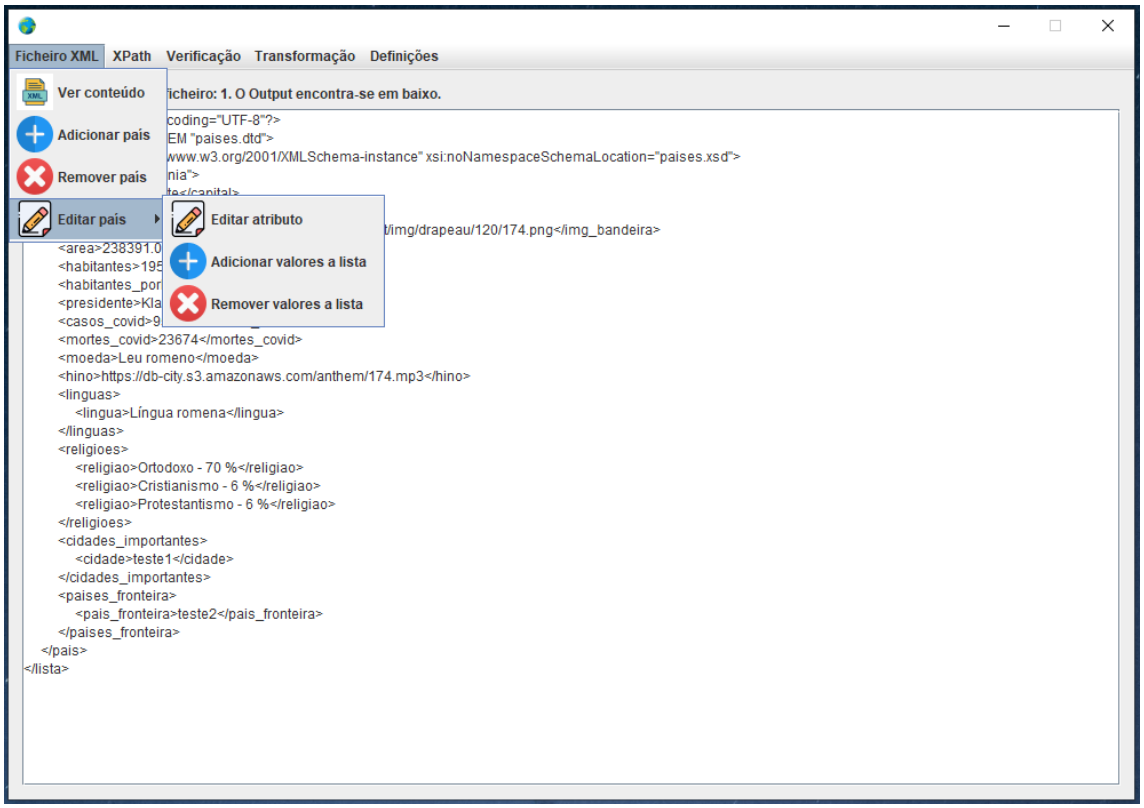


As duas opções das definições tratam-se de implementar o "Modo Escuro", de forma a toda a aplicação ficar com fundo escuro e letras claras, bem como de sair da aplicação.

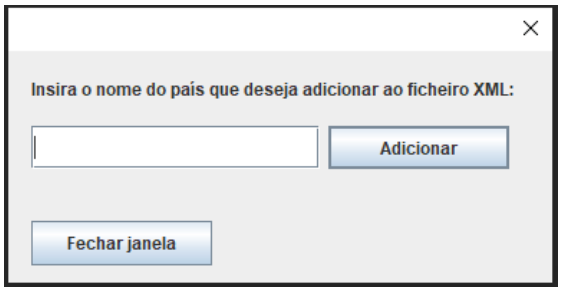


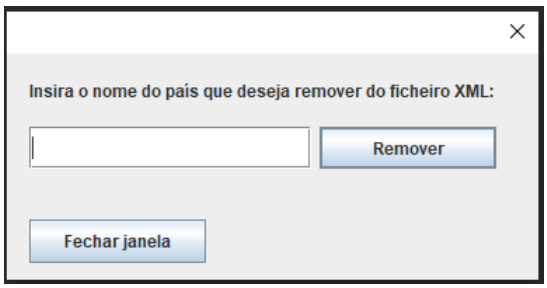
2.2. Manipulação ao ficheiro XML

Nesta primeira opção o utilizador consegue ver o conteúdo do ficheiro XML (caso já não esteja a mostrar na caixa de texto principal), adicionar um país ao ficheiro, remover um país do ficheiro, editar um atributo (não array, e.g. capital, moeda etc.) e adicionar e remover valores a listas (arrays, e.g. línguas, religiões, etc.).

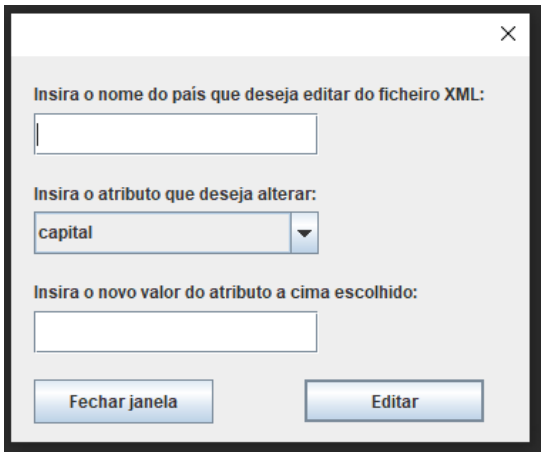


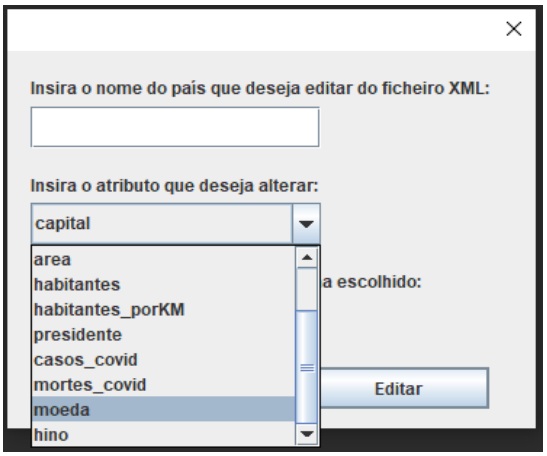
Como se pode observar foram criados 2 jDialogs para cada uma das opções (adicionar e remover um país) para assim o utilizador inserir o nome do país a adicionar, bem como a remover.



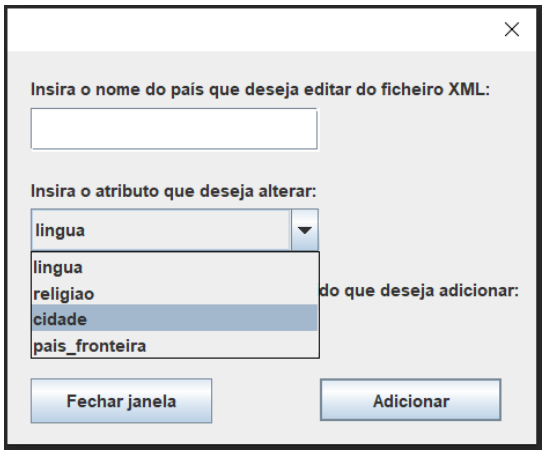


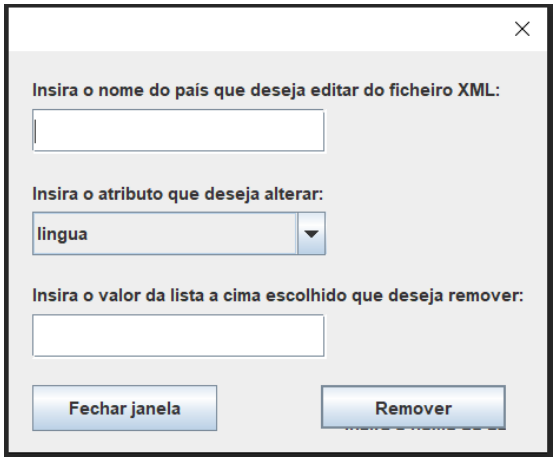
Quando se tenta editar um atributo de um país qualquer, ao diferente do pedido no enunciado, o nosso projeto dá a oportunidade ao utilizador de escolher qualquer atributo a editar (nesta aba estão incluídos todos os atributos que não são listas). Seguem duas imagens a ilustrar o funcionamento da jDialog de editar o país.





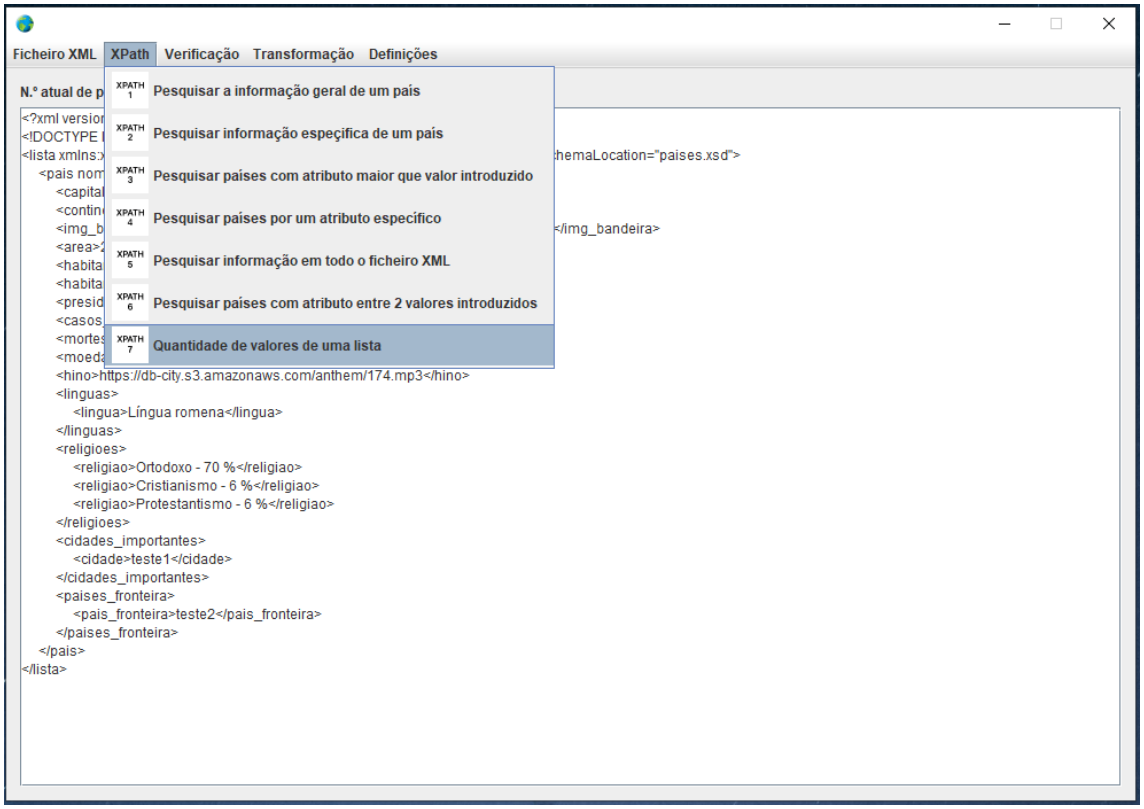
Quando se trata de adicionar/remover valores dos atributos listas dos países, o nosso projeto também dá a oportunidade ao utilizador de escolher qual desses atributos que deseja adicionar ou remover valores. Seguem duas imagens a ilustrar as duas opções de adicionar e remover valores a atributos que são listas.



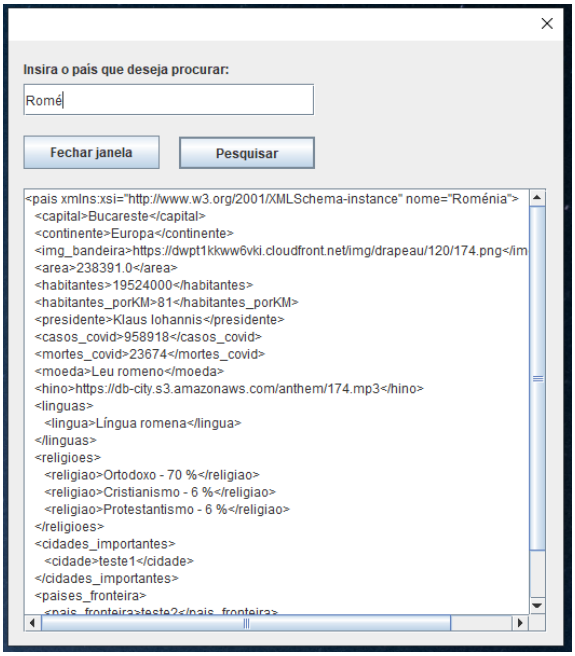


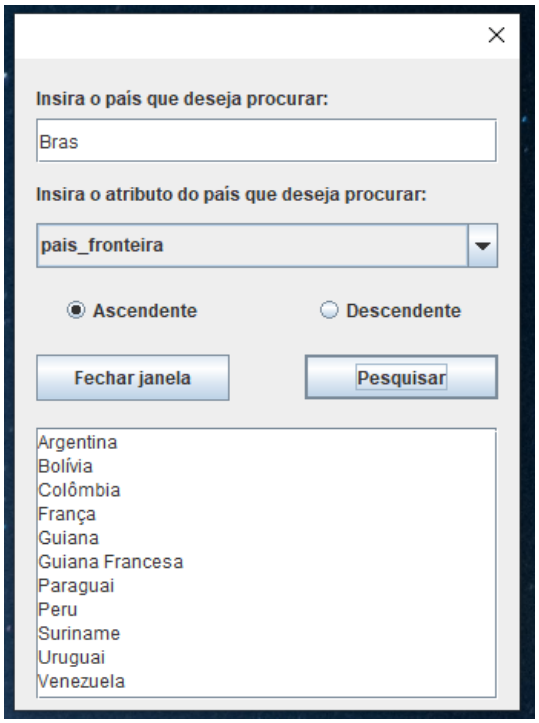
2.3. Pesquisas ao ficheiro XML com XPath

Há sete submenus na opção "XPath", sendo cada uma delas uma pesquisa XPath. Das sete pesquisas seis são relacionadas com as do enunciado (dizemos relacionadas pois em vez de ser sobre apenas e só um atributo, o utilizador pode mais uma vez selecionar o atributo que quer pesquisar ou pelo qual quer pesquisar) e uma é opcional. Na imagem seguinte está a ser ilustrado o submenu "XPath":



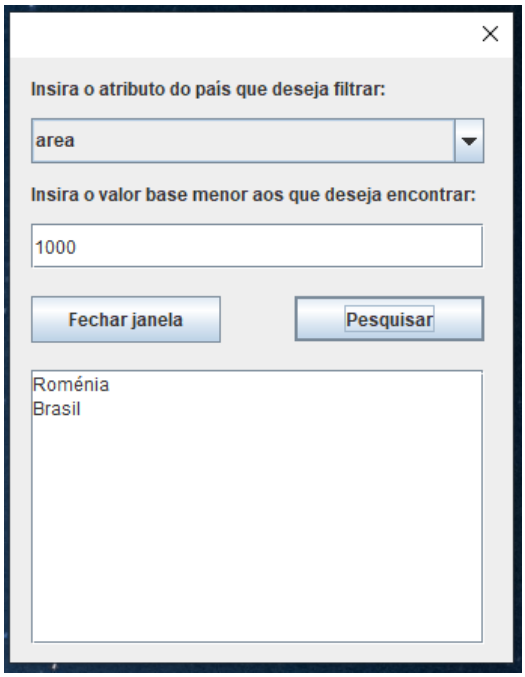
1ª pesquisa XPath: Pesquisar a informação geral de um país – Esta pesquisa mostra todos os atributos do país inserido pelo utilizador. O nome do país não necessita de estar completo, não dando tanto trabalho ao utilizador.

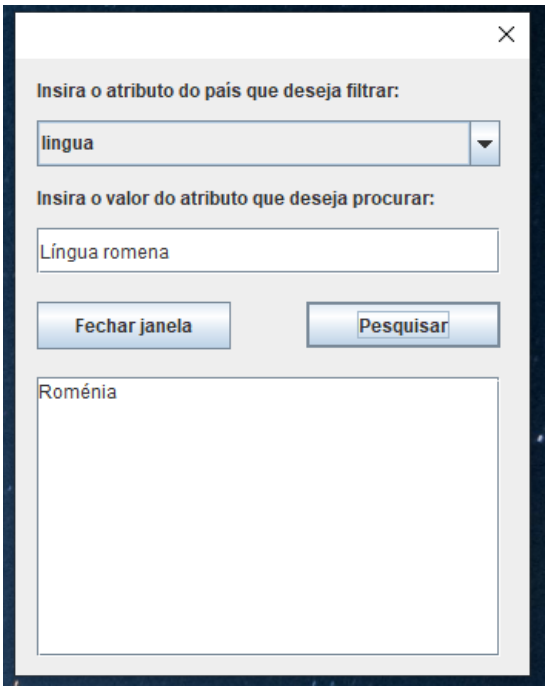




2ª pesquisa XPath: Pesquisar informação específica de um país – Nesta pesquisa o utilizador apenas necessita de inserir o nome do país (ou parte dele), escolher qual a informação do mesmo que deseja observar (tanto atributos não lista como lista) e ainda pode selecionar a ordenação do mesmo (ascendente ou descendente).

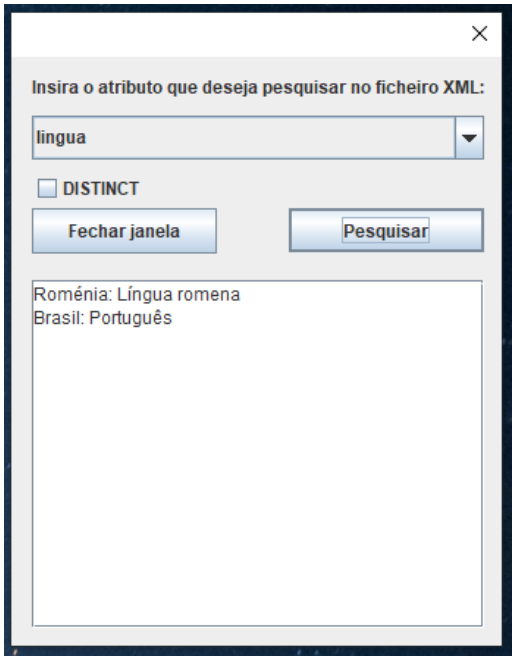
3ª pesquisa XPath: Pesquisar países com atributo maior que valor introduzido – Nesta pesquisa o utilizador começa por escolher qual o atributo que deseja filtrar (apenas atributos não lista e numerários, pois aqui está-se a mexer com esse tipo de valores), insere o valor base que deseja ser inferior ao resto e faz a sua pesquisa.

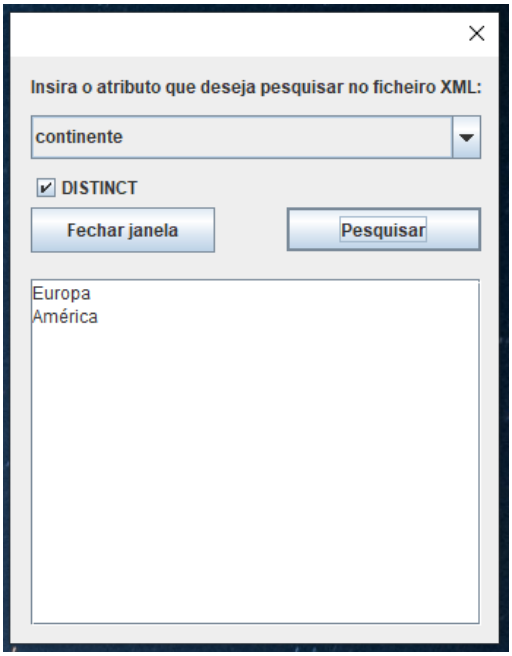




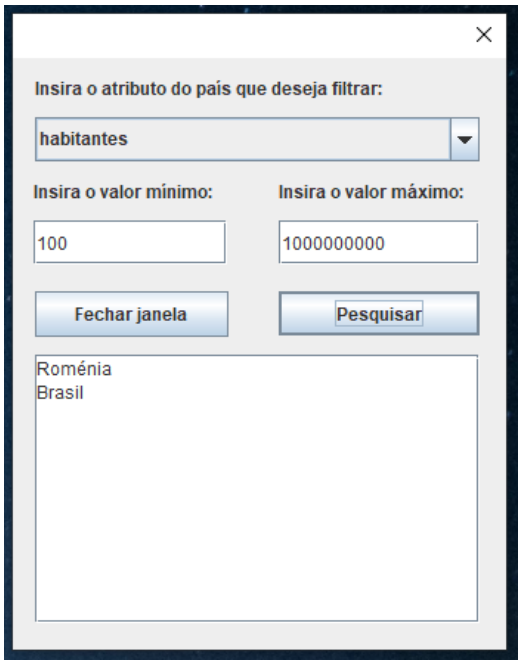
4ª pesquisa XPath: Pesquisar países por um atributo específico – Nesta pesquisa o utilizador começa por escolher qual o atributo que deseja filtrar (qualquer atributo, seja ele lista ou não lista ou até numerário ou não numerário), insere o valor do atributo que deseja pesquisar, faz a sua pesquisa e aparece-lhe os países que têm aquele valor no atributo em questão. Nesta pesquisa XPath o utilizador se escolher um atributo não numerário (seja ele lista ou não lista) pode escrever parte do valor e não o valor todo, ao contrário do caso se for um atributo numerário pois não faz sentido numa área ser parte do valor da mesma (e.g.).

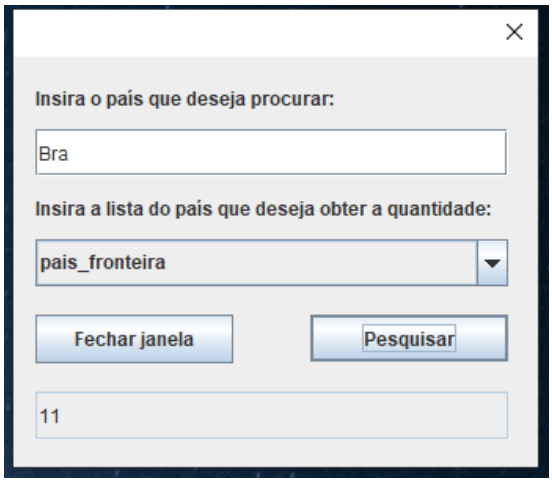
5ª pesquisa XPath: Pesquisar informação em todo o ficheiro XML – Esta pesquisa vai buscar toda a informação do atributo escolhido pelo utilizador ao ficheiro XML, podendo ainda escolher a opção "DISTINCT", onde basicamente o programa elimina do *output* os valores repetidos (e.g. caso o utilizador queira observar os continentes do ficheiro pode optar por não os ver a repetir (caso houvesse +1 país num continente específico)). Caso essa opção não esteja marcada cada informação aparece com o devido país à sua esquerda de forma a o utilizador saber a que país essa informação se refere (isto, porque no exemplo de cima não fazia sentido aparecer apenas um país e o nome do continente que é de mais países).





6ª pesquisa XPath: Pesquisar países com atributo entre 2 valores introduzidos – Novamente nesta pesquisa XPath o utilizador pode inserir qual o atributo que deseja filtrar, introduzir ambos os valores, pesquisar e observar os resultados da sua pesquisa.

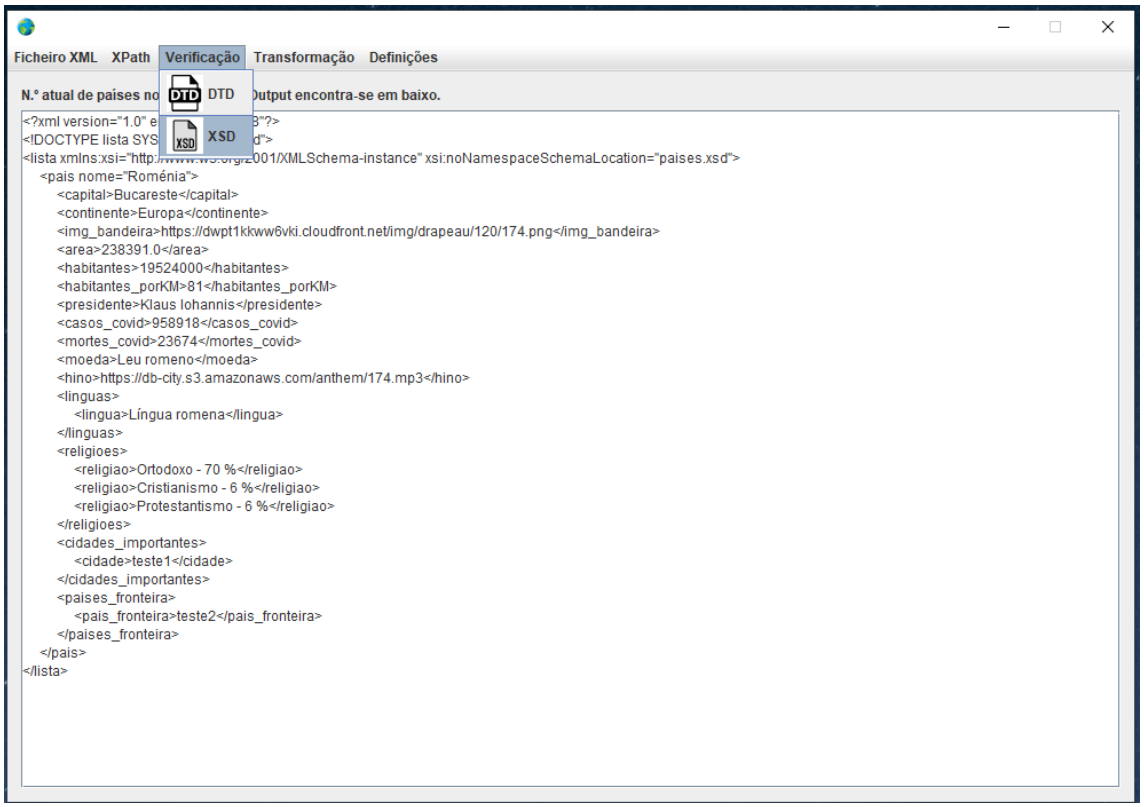




7ª pesquisa XPath: Quantidade de valores de uma lista – Esta pesquisa XPath é totalmente opcional onde basicamente o utilizador apenas insere o nome de um país (ou parte dele), seleciona o atributo (apenas de lista) que deseja saber a quantidade, pesquisa e observa a quantidade desses atributos de lista pesquisados do país também inserido.

2.4. Verificação com DTD

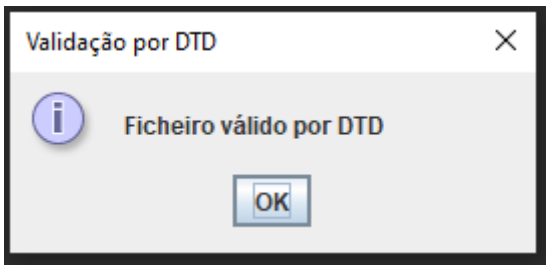
Na terceira opção do menu principal estão duas opções de verificação ao ficheiro XML principal: DTD e XSD. Ilustra-se essas mesmas opções na imagem a seguir:



De seguida ilustra-se também o código do ficheiro DTD que valida o ficheiro XML:

```
1   <!ELEMENT lista  ( pais+ )>
2
3   <!ELEMENT pais  ( capital , continente , img_bandeira , area , habitantes , habitantes_porKM , presidente , casos_covid , mortes_covid , moeda , hino , linguas , religioes , cidades_importantes , paises_fronteira )>
4
5   <!ATTLIST pais nome CDATA #REQUIRED>
6
7   <!ELEMENT capital  ( #PCDATA )>
8
9   <!ELEMENT continente  ( #PCDATA )>
10
11  <!ELEMENT img_bandeira  ( #PCDATA )>
12
13  <!ELEMENT area  ( #PCDATA )>
14
15  <!ELEMENT habitantes  ( #PCDATA )>
16
17  <!ELEMENT habitantes_porKM  ( #PCDATA )>
18
19  <!ELEMENT presidente  ( #PCDATA )>
20
21  <!ELEMENT casos_covid  ( #PCDATA )>
22
23  <!ELEMENT mortes_covid  ( #PCDATA )>
24
25  <!ELEMENT moeda  ( #PCDATA )>
26
27  <!ELEMENT hino  ( #PCDATA )>
28
29  <!ELEMENT linguas  ( lingua* )>
30
31  <!ELEMENT lingua  ( #PCDATA )>
32
33  <!ELEMENT religioes  ( religiao* )>
34
35  <!ELEMENT religiao  ( #PCDATA )>
36
37  <!ELEMENT cidades_importantes  ( cidade* )>
38
39  <!ELEMENT cidade  ( #PCDATA )>
40
41  <!ELEMENT paises_fronteira  ( pais_fronteira* )>
42
43  <!ELEMENT pais_fronteira  ( #PCDATA )>
44
45  <!ATTLIST lista xmlns:xsi CDATA #IMPLIED>
46
47  <!ATTLIST lista xsi:noNamespaceSchemaLocation CDATA #IMPLIED>
```

De seguida também se pode ver o resultado da validação com DTD na aplicação:



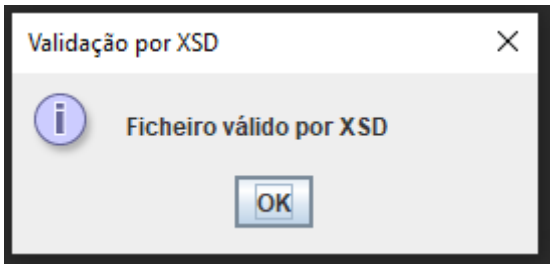
2.5. Verificação com XSD

De seguida ilustra-se o código do ficheiro XSD que valida o ficheiro XML:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema elementFormDefault="qualified" xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="lista">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="pais" maxOccurs="unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
    <xs:element name="continente" type="xs:string"/>
    <xs:element name="img_bandeira" type="xs:string"/>
    <xs:element name="area" type="xs:string"/>
    <xs:element name="cidades_importantes">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="cidade" maxOccurs="unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
    <xs:element name="capital" type="xs:string"/>
    <xs:element name="cidade" type="xs:string"/>
    <xs:element name="habitantes" type="xs:string"/>
    <xs:element name="lingua" type="xs:string"/>
    <xs:element name="paises_fronteira">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="pais_fronteira" maxOccurs="unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
    <xs:element name="habitantes_porKM" type="xs:string"/>
    <xs:element name="casos_covid" type="xs:string"/>
    <xs:element name="pais">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="capital"/>
                <xs:element ref="continente"/>
                <xs:element ref="img_bandeira"/>
                <xs:element ref="area"/>
                <xs:element ref="habitantes"/>
                <xs:element ref="habitantes_porKM"/>
                <xs:element ref="presidente"/>
                <xs:element ref="casos_covid"/>
                <xs:element ref="mortes_covid"/>
                <xs:element ref="moeda"/>
```

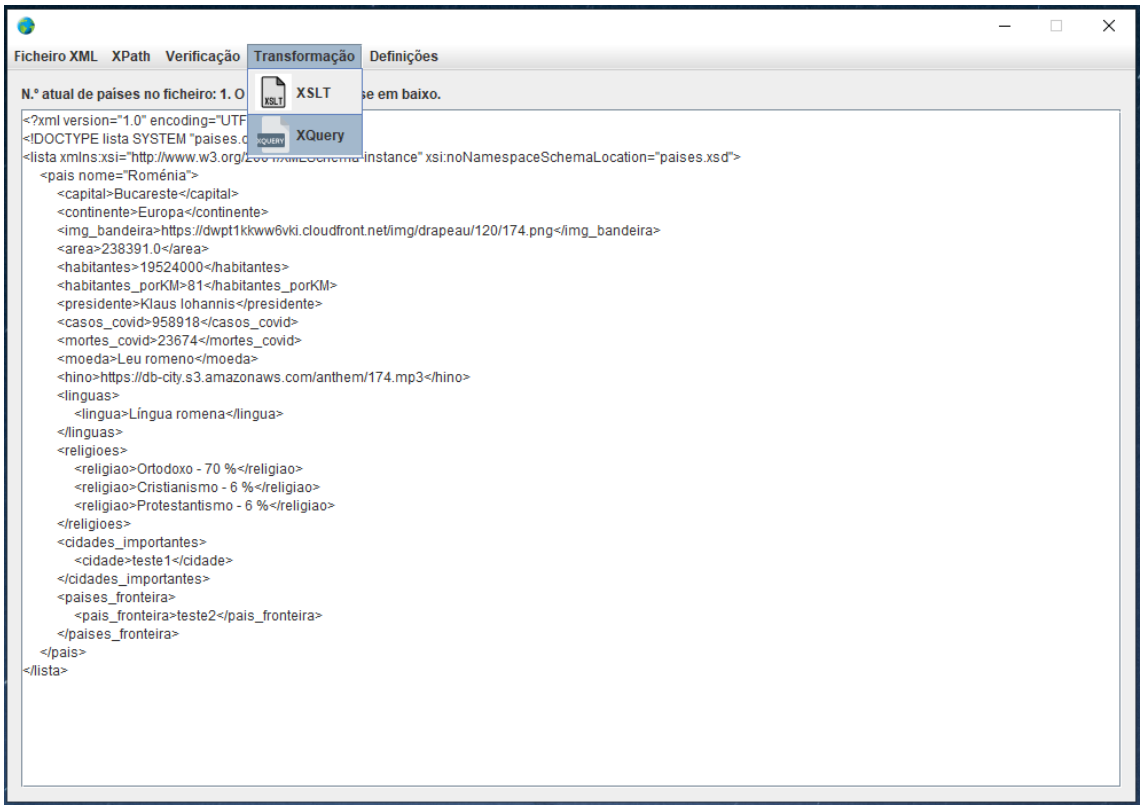
```xml
<xs:schema elementFormDefault="qualified" xmlns:xs="http://www.w3.org/20
    <xs:element name="casos_covid" type="xs:string"/>
    <xs:element name="pais">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="capital"/>
                <xs:element ref="continente"/>
                <xs:element ref="img_bandeira"/>
                <xs:element ref="area"/>
                <xs:element ref="habitantes"/>
                <xs:element ref="habitantes_porKM"/>
                <xs:element ref="presidente"/>
                <xs:element ref="casos_covid"/>
                <xs:element ref="mortes_covid"/>
                <xs:element ref="moeda"/>
                <xs:element ref="hino"/>
                <xs:element ref="linguas"/>
                <xs:element ref="religioes"/>
                <xs:element ref="cidades_importantes"/>
                <xs:element ref="paises_fronteira"/>
            </xs:sequence>
            <xs:attribute name="nome" type="xs:string" use="required"/>
        </xs:complexType>
    </xs:element>
    <xs:element name="religioes">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="religiao" maxOccurs="unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
    <xs:element name="mortes_covid" type="xs:string"/>
    <xs:element name="hino" type="xs:string"/>
    <xs:element name="moeda" type="xs:string"/>
    <xs:element name="pais_fronteira" type="xs:string"/>
    <xs:element name="linguas">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="lingua" maxOccurs="unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
    <xs:element name="presidente" type="xs:string"/>
    <xs:element name="religiao" type="xs:string"/>
</xs:schema>
```

De seguida também se pode ver o resultado da validação com XSD na aplicação:

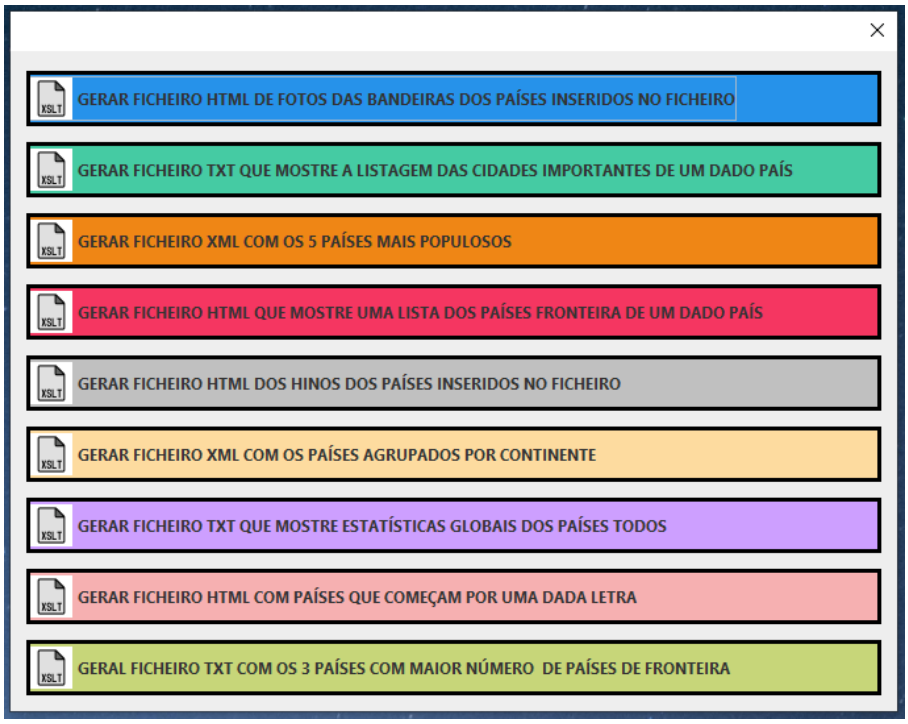


2.6. Transformação com XSLT

Na quarta opção do menu principal estão novamente duas opções, só que desta vez de transformação a partir do ficheiro XML principal: XSLT e XQuery. Ilustra-se essas mesmas opções na imagem a seguir:



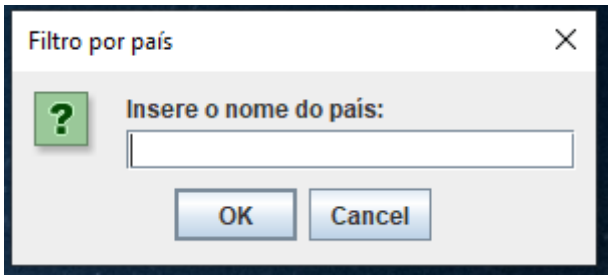
Quando se clica na opção de transformação XSLT, o programa abre um jDialog, eis o mesmo:



Basicamente o nosso programa oferece de uma a nove transformações:

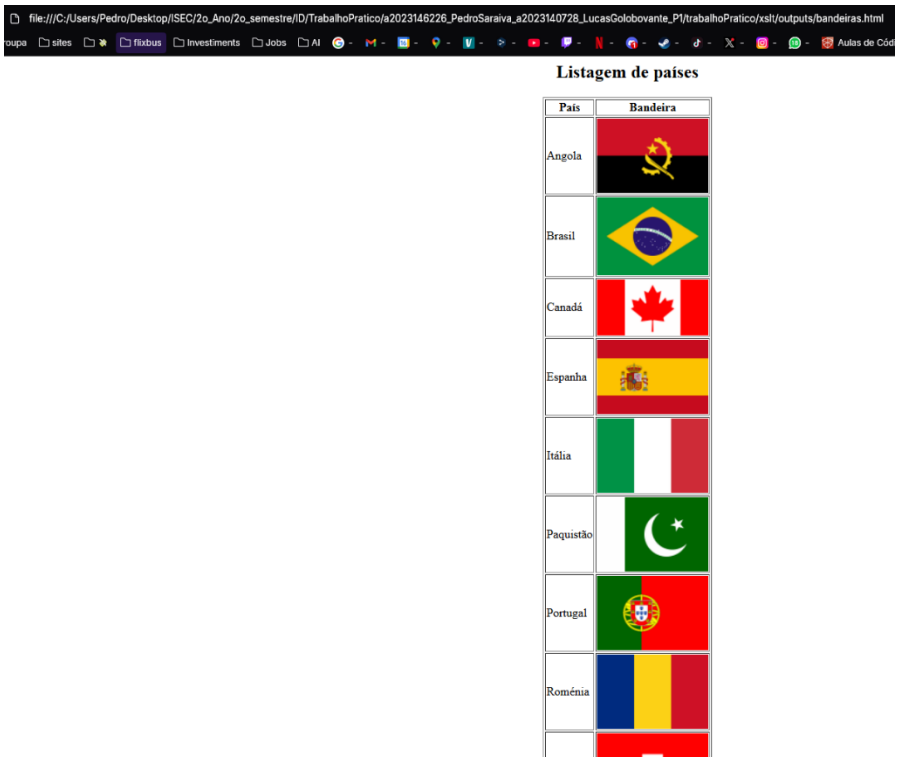
- 4 mencionadas no enunciado;
- 3 obrigatórias criadas por nós;
- 2 opcionais criadas por nós.

Nas transformações onde é necessário a inserção de algo do utilizador (seja um nome dum país, uma letra, etc.) é mostrada a seguinte janela de inserção ao utilizador:



De seguida vai-se ilustrar os outputs das devidas transformações em XSLT:

- 1ª transformação: Gerar ficheiro HTML de fotos das bandeiras dos países inseridos no ficheiro:



- 2ª transformação: Gerar ficheiro TXT que mostre a listagem das cidades importantes de um dado país:

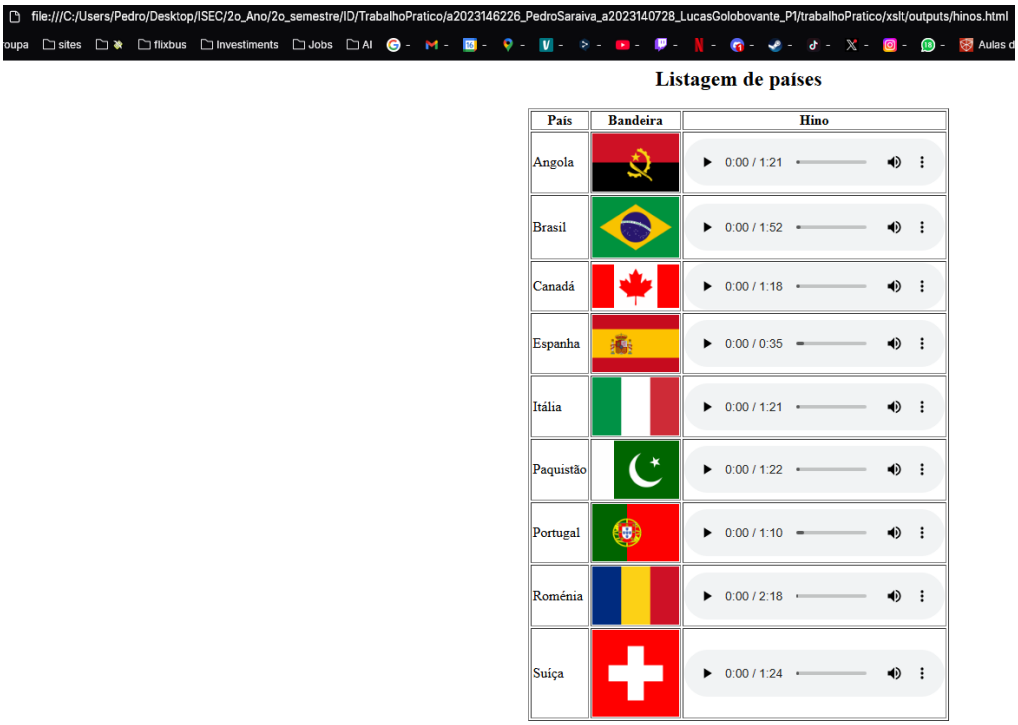
- 3ª transformação: Gerar ficheiro XML com os 5 países mais populosos:

- 4ª transformação: Gerar ficheiro HTML que mostre uma lista dos países fronteira de um dado país:

Países de fronteira d(a/e/o) Brasil

- Argentina
- Bolívia
- Colômbia
- França
- Guiana
- Guiana Francesa
- Paraguai
- Peru
- Suriname
- Uruguai
- Venezuela

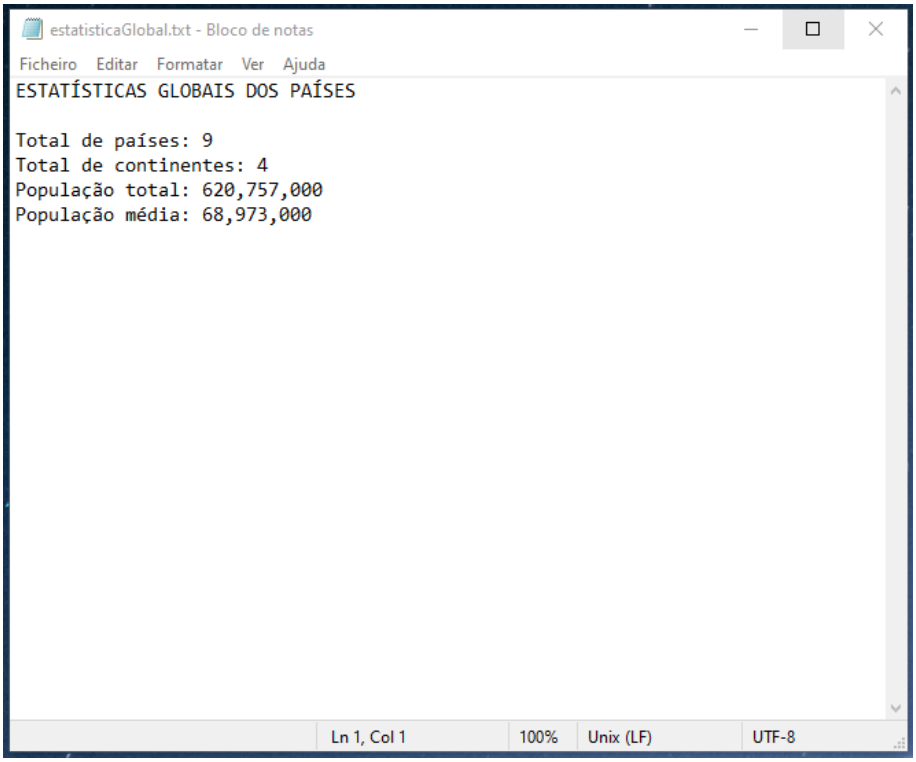
- 5ª transformação: Gerar ficheiro HTML dos hinos dos países inseridos no ficheiro:



- 6ª transformação: Gerar ficheiro XML com os países agrupados por continente:

```xml
▼<paisesPorContinente>
  ▼<continente nome="Europa">
      <pais>Roménia</pais>
      <pais>Portugal</pais>
      <pais>Suíça</pais>
      <pais>Itália</pais>
      <pais>Espanha</pais>
    </continente>
  ▼<continente nome="América">
      <pais>Brasil</pais>
      <pais>Canadá</pais>
    </continente>
  ▼<continente nome="Ásia">
      <pais>Paquistão</pais>
    </continente>
  ▼<continente nome="África">
      <pais>Angola</pais>
    </continente>
  </paisesPorContinente>
```

- 7ª transformação: Gerar ficheiro TXT que mostre estatísticas globais dos países todos:

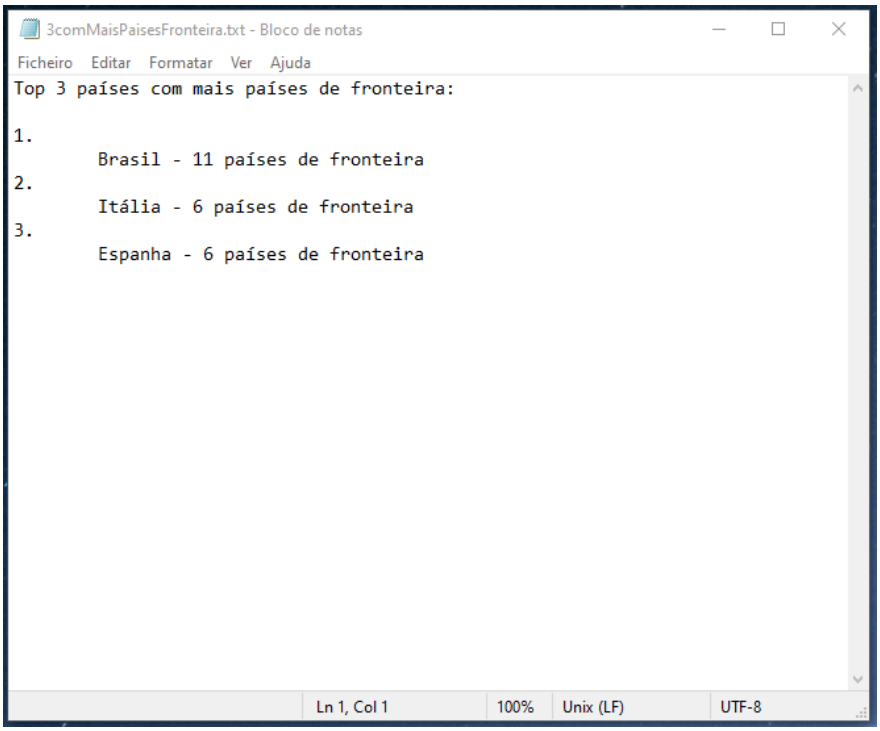


- 8ª transformação: Gerar ficheiro HTML com países que começam por uma dada letra:

Países que começam por 'P'

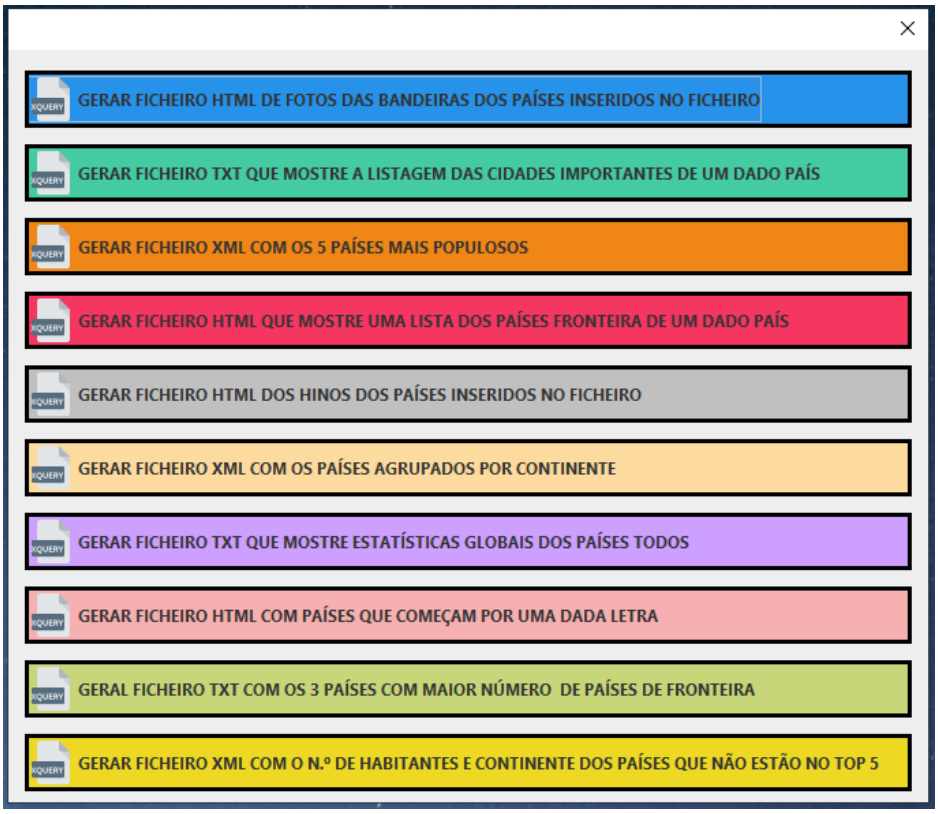
- Paquistão
- Portugal

- 9ª transformação: Gerar ficheiro TXT com os 3 países com maior número de países de fronteira:



2.7. Transformação com XQuery

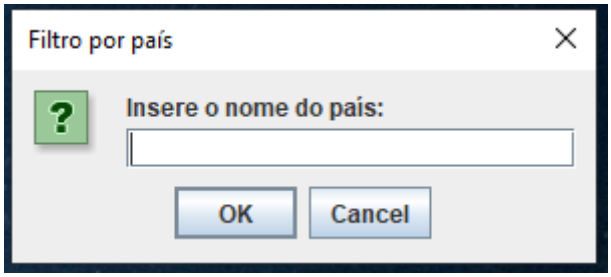
Quando se clica na opção de transformação XQuery, o programa abre um jDialog, eis o mesmo:



Basicamente o nosso programa oferece de uma a nove transformações:

- 4 mencionadas no enunciado;
- 3 obrigatórias criadas por nós;
- 3 opcionais criadas por nós.

Tal como no XSLT, nas transformações onde é necessário a inserção de algo do utilizador (seja um nome dum país, uma letra, etc.) é mostrada a seguinte janela de inserção ao utilizador:



Visto que os outputs das primeiras 9 transformações em XQuery são iguais aos outputs das 9 primeiras transformações em XSLT, apenas iremos mostrar a seguir o output da 10ª transformação em XQuery, sendo importante ter em consideração a 3ª e a 6ª transformação.

A 10ª transformação junta os dados dos ficheiros XML da 3ª transformação e da 6ª transformação e ainda do ficheiro XML principal ("países.xml"). Como já foi falado anteriormente a 3ª transformação extrai os 5 países mais populosos enquanto a 6ª transformação extrai os países por continente. Desta forma o ficheiro XML da 10ª transformação extrai os países que estão no ficheiro XML da 6ª transformação e que não estão no ficheiro da 3ª transformação, extraindo por tanto todos os países que não estão no top 5 e que existem no ficheiro da 6ª transformação. A informação a ser extraída é: o **nome do país** (ficheiro XML da 6ª transformação); o **número de habitantes** (ficheiro XML principal); e por fim o **continente** onde fica o país (ficheiro XML da 6ª transformação)

Eis o output da 10ª transformação com XQuery: Gerar ficheiro XML com o n.º de habitantes e continente dos países que não estão no top 5:

```
▼<informacaoJunta>
  ▼<pais>
      <nome>Angola</nome>
      <habitantes>29250000</habitantes>
      <continente>África</continente>
    </pais>
  ▼<pais>
      <nome>Roménia</nome>
      <habitantes>19524000</habitantes>
      <continente>Europa</continente>
    </pais>
  ▼<pais>
      <nome>Portugal</nome>
      <habitantes>10287000</habitantes>
      <continente>Europa</continente>
    </pais>
  ▼<pais>
      <nome>Suíça</nome>
      <habitantes>8484000</habitantes>
      <continente>Europa</continente>
    </pais>
</informacaoJunta>
```

3. Conclusão

Nós de uma forma geral gostámos muito de fazer o trabalho, é claro que nem sempre o desenvolvimento se alinhou com o objetivo, por exemplo na extração da informação dos dados nas páginas web, como já era previsto, principalmente na Wikipédia, a informação divergia bastante de país para país, sendo necessário aprimorar de pouco a pouco os Wrappers.

Adquirimos também muitas competências mais precisamente em extração de dados a páginas web com Wrappers, manipulação de ficheiros XML, pesquisas XPath, verificação a ficheiros XML com DTD e XSD e ainda extrações para outro tipo de ficheiros com XLST e XQuery.